University of Sialkot

Semester Project – Deliverable 2 (Client-Side + Server Side Architecture)

STUDENT ROLL NUMBER NAME Sajawal Shahbaz (23010101-049)
Ehtesham Hafeez (23010101-094)

TOTAL GRADE POINT 10

GRADE POINT ALLOCATED

ASSESSOR'S COMMENTS

Instructor: Museb Khalid

Medical Report Analyzer

What this project is?

Medical Report Analyzer is a dashboard-style app that helps someone keep track of their medical reports blood tests, X-rays, MRIs, ECGs in one place instead of digging through paper files or random PDFs every time. A user uploads a report along with basic patient details, and the app is meant to show a clean summary of the important numbers (blood pressure, sugar level, heart rate, oxygen level) and keep a history of everything that was uploaded before.

This deliverable is only about the frontend. The actual analysis logic, authentication, and storage will be connected later once the backend (Express + MongoDB) is added in the next phase, so right now the app just runs on sample/local data to prove the interface and navigation work properly.

1. Frontend (Built with Vue 3 + Vite):

The frontend is a single-page application built with Vue 3 and Vite. Instead of separate HTML pages for every screen, the whole thing is one app that swaps out components on the fly, which is what makes it feel instant when moving between pages.

1.1 Vue Router Or Page Navigation:

Vue Router handles moving between all the main pages Home, Dashboard, Login, Register, Upload, Analysis, History and Profile without ever reloading the browser. Every route is registered in one place, src/router/index.js, and I used history mode so the URLs look normal (like /dashboard) instead of having a # in them.

The layout itself lives in App.vue, where the Navbar, Sidebar and Footer stay fixed on every page, and only the middle part changes using <router-view>, wrapped in a transition so pages fade in instead of just popping in.

1.2 Component Structure:

The project is split into two folders. components/ holds pieces that get reused Navbar, Sidebar, Footer, DashboardChart, FeatureCard, CustomButton and views/ holds one file per page/route

(Home, Dashboard, Login, Register, Upload, Analysis, History, Profile, NotFound). This keeps every view focused on just its own page, and the shared stuff like the navbar never has to be repeated.

1.3 Props and Emits:

To actually show data flowing between a parent and a child component, I built two small reusable components. FeatureCard.vue just displays whatever it's given an icon, a title and a description are passed in as props from the parent, so the card itself doesn't need to know anything, it just renders what it receives.

CustomButton.vue works the other way around. It doesn't decide what happens when it's clicked it just emits a button-click event and lets the parent decide what to do with it. That way the same button can be reused anywhere and just wired up differently each time.

Put together, a parent view would use them like this passing data down through props, and listening for the click going back up:

1.4 Other Reusable Components:

DashboardChart.vue wraps a Chart.js bar chart and is dropped straight into the Dashboard page to show monthly report counts. Navbar.vue, Sidebar.vue and Footer.vue are the shell pieces that stay on screen no matter which page is active, so they only ever get written once.

1.5 Local State:

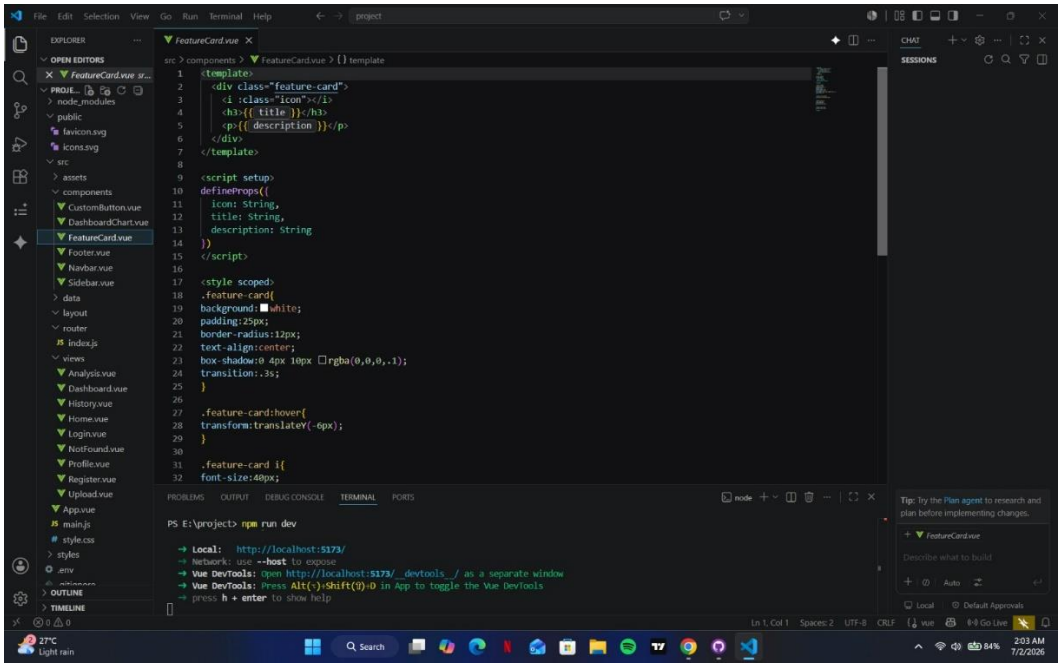
Anything that doesn't need to be shared across components is just kept as local state using Vue's ref(). For example, Navbar.vue keeps its own dark-mode toggle in a ref, and Upload.vue keeps every form field (patient name, age, gender, report type, hospital, doctor, notes) as its own ref bound with v-model, only collecting everything together when the form is submitted.

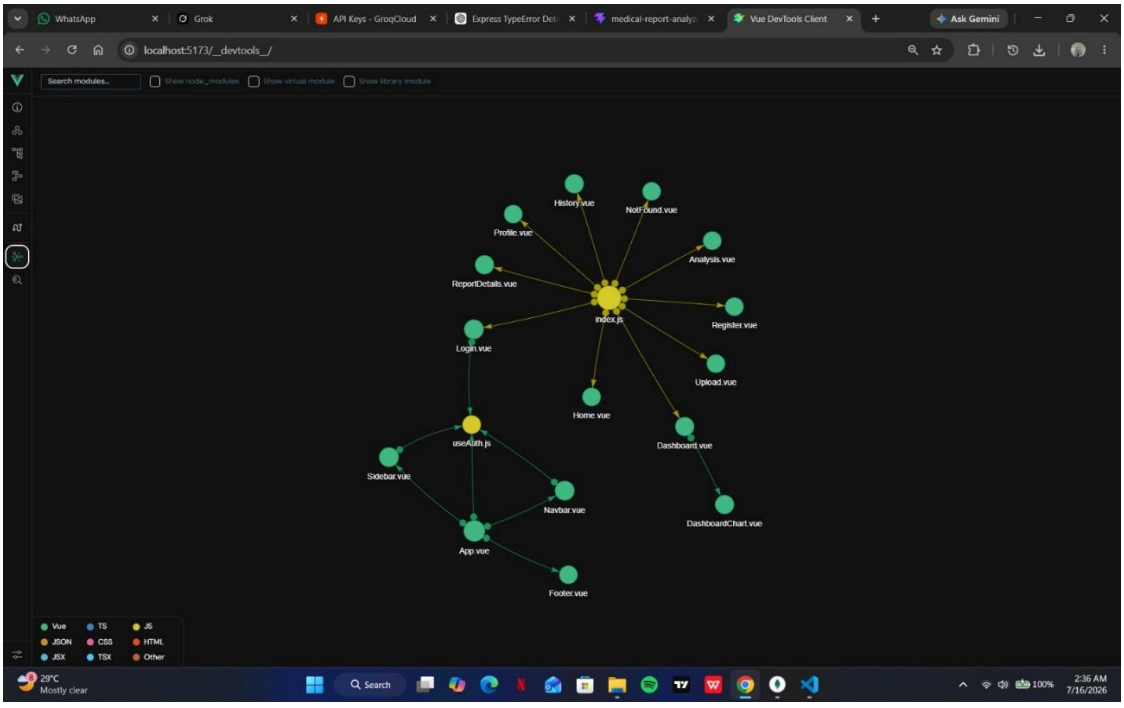
1.6 Styling:

Styling is done with a shared global CSS file imported once in main.js, so every page and component ends up looking consistent without repeating the same colors and spacing everywhere.

1.7 Component Architecture:

Here's a simple diagram of how the components fit together the shell (Navbar, Sidebar, Footer) sits around whatever page the router is currently showing, and the reusable widgets (FeatureCard, CustomButton, DashboardChart) can be dropped into any view that needs them.

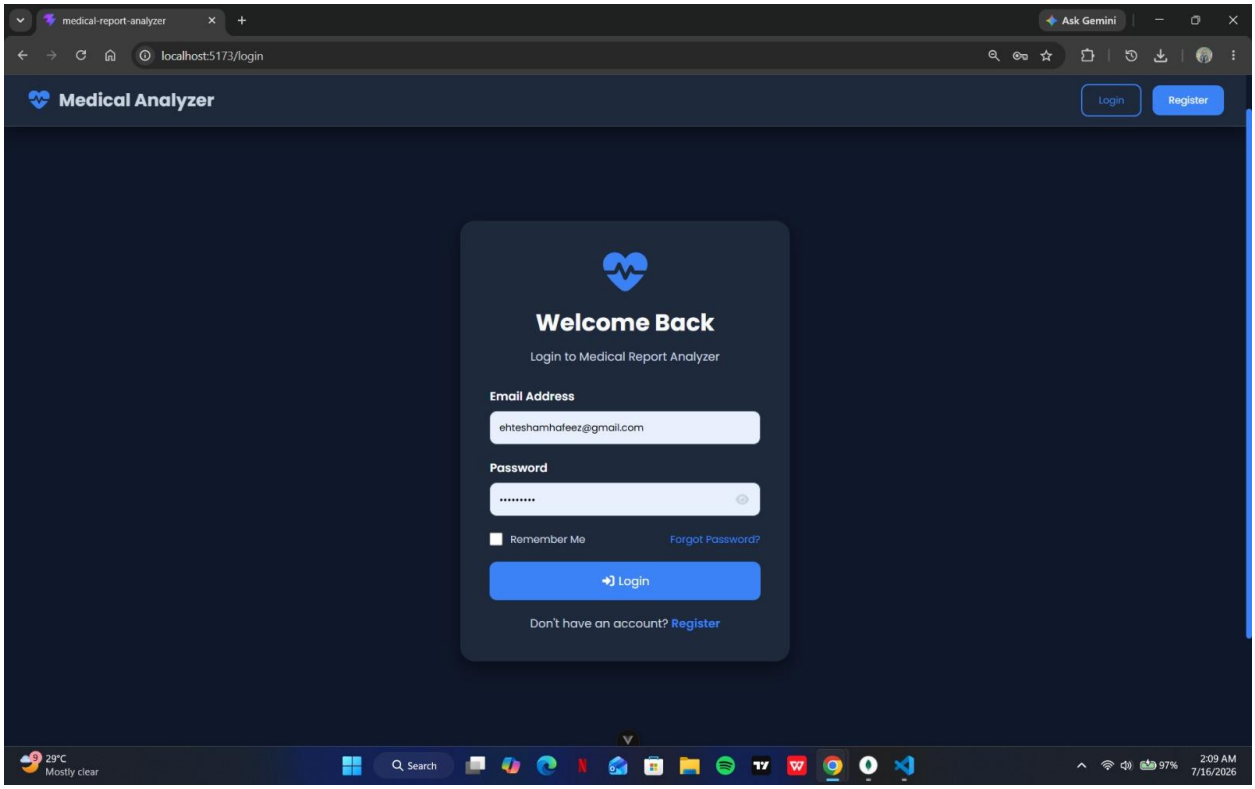




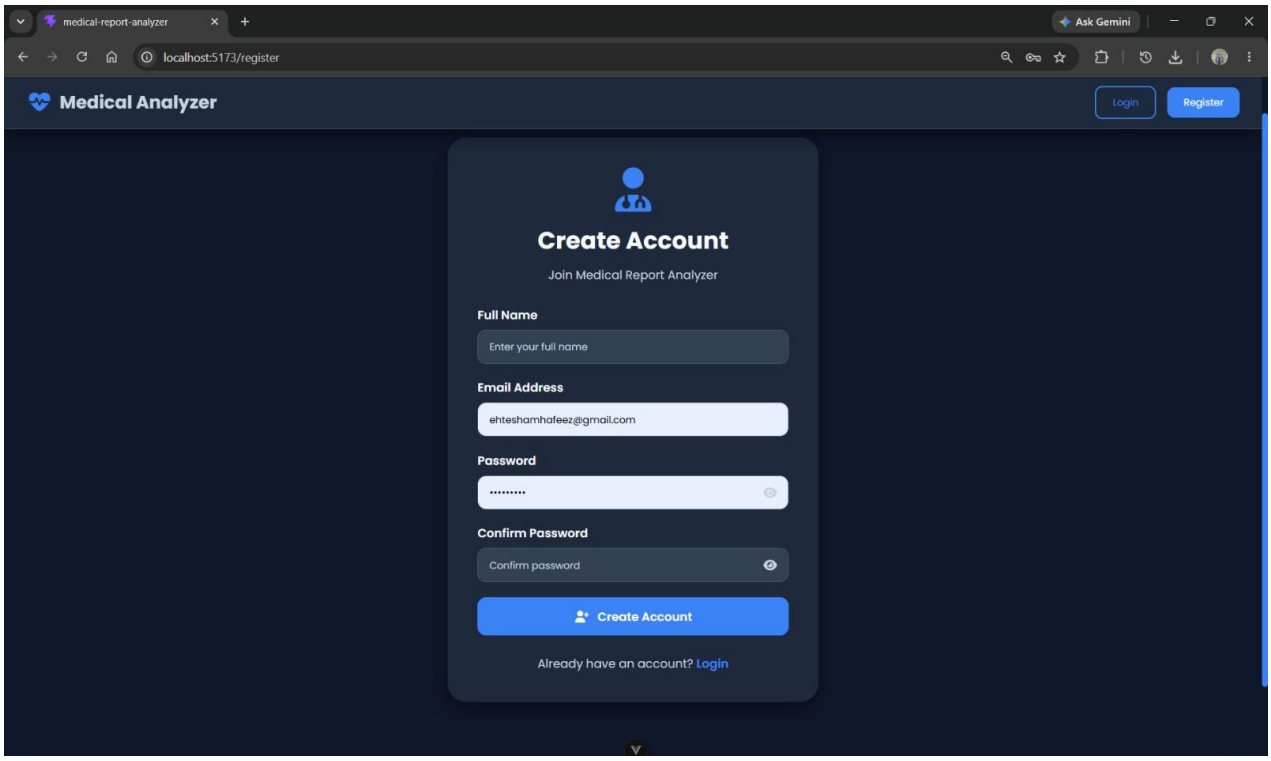
2 UI Screenshots:

The app was run locally with npm run dev, which serves it at http://localhost:5173/. These screenshots show the interface actually running.

2.1 Login page:

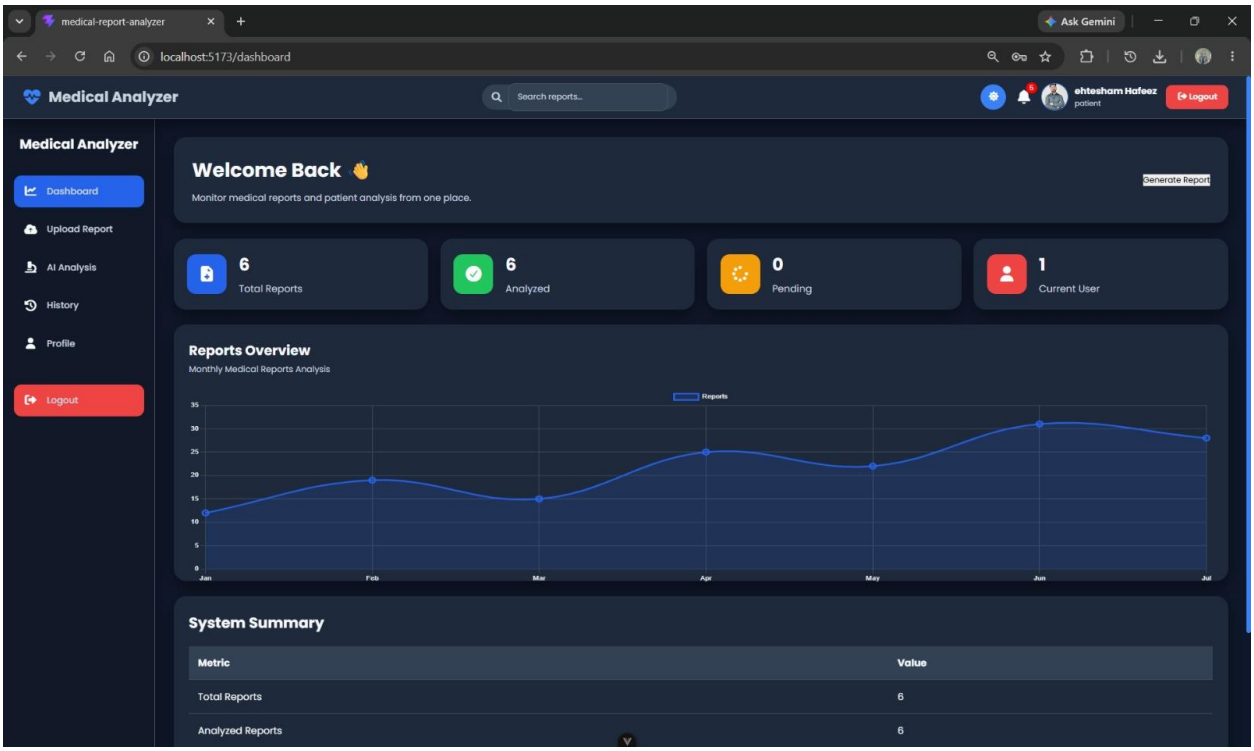


2.2 Register Page:

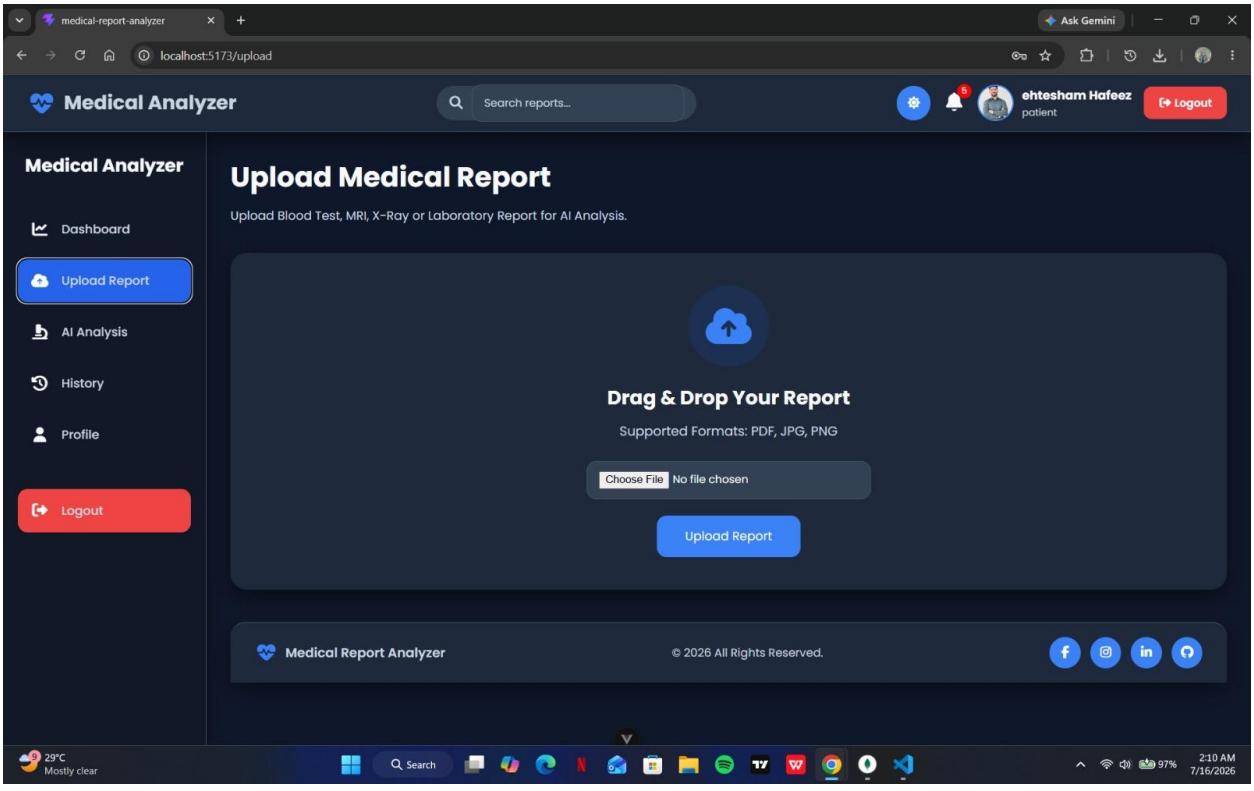


2.3 Home Page:

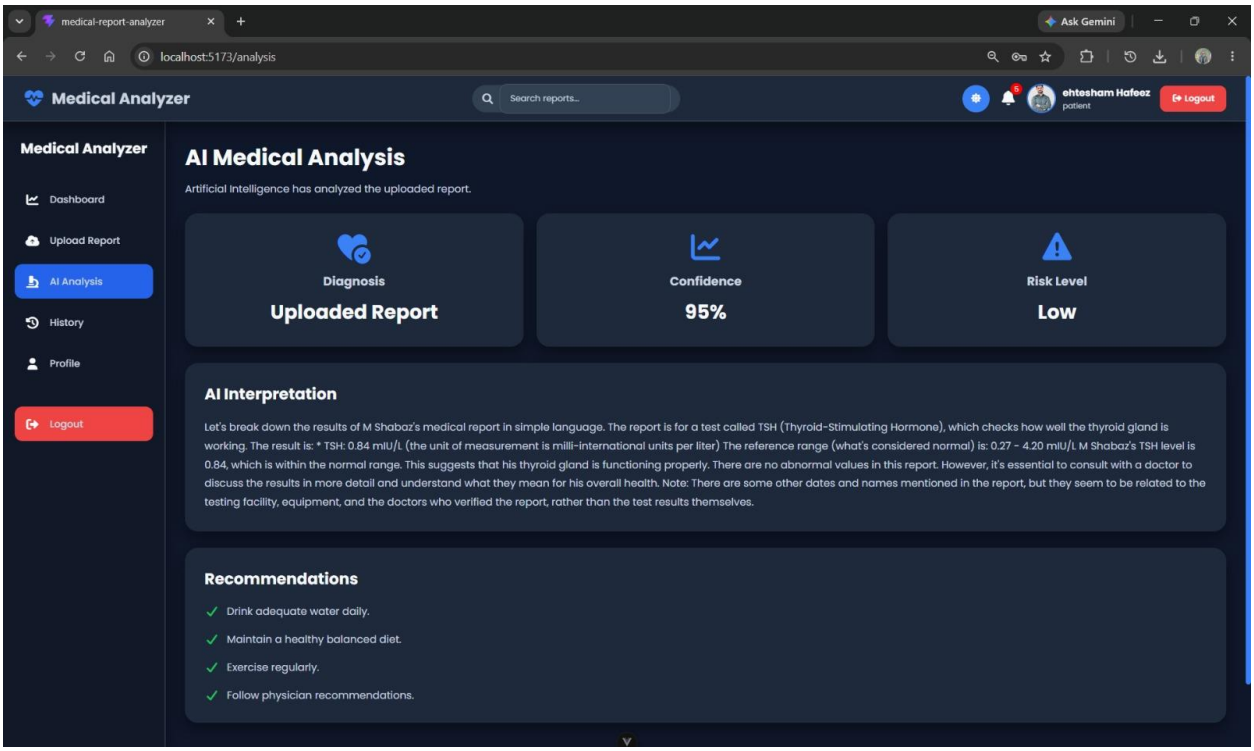
2.4 Dashboard Page:



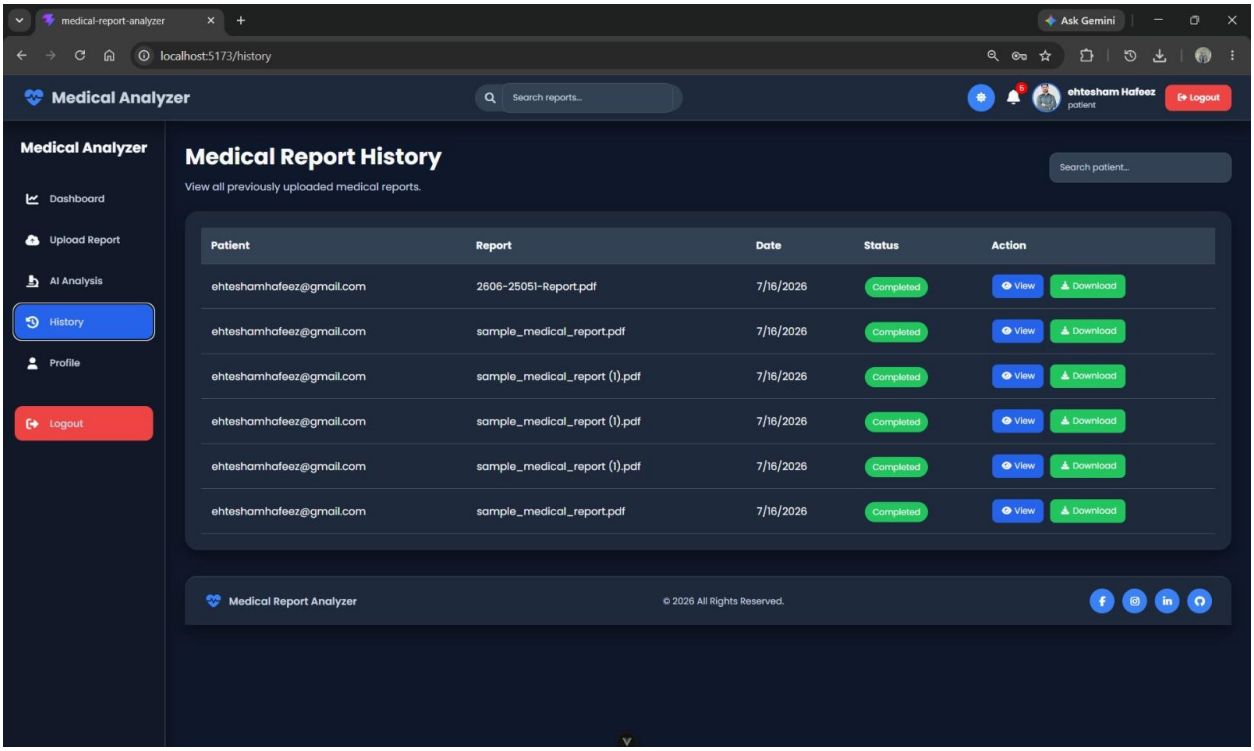
2.5 Upload Page:



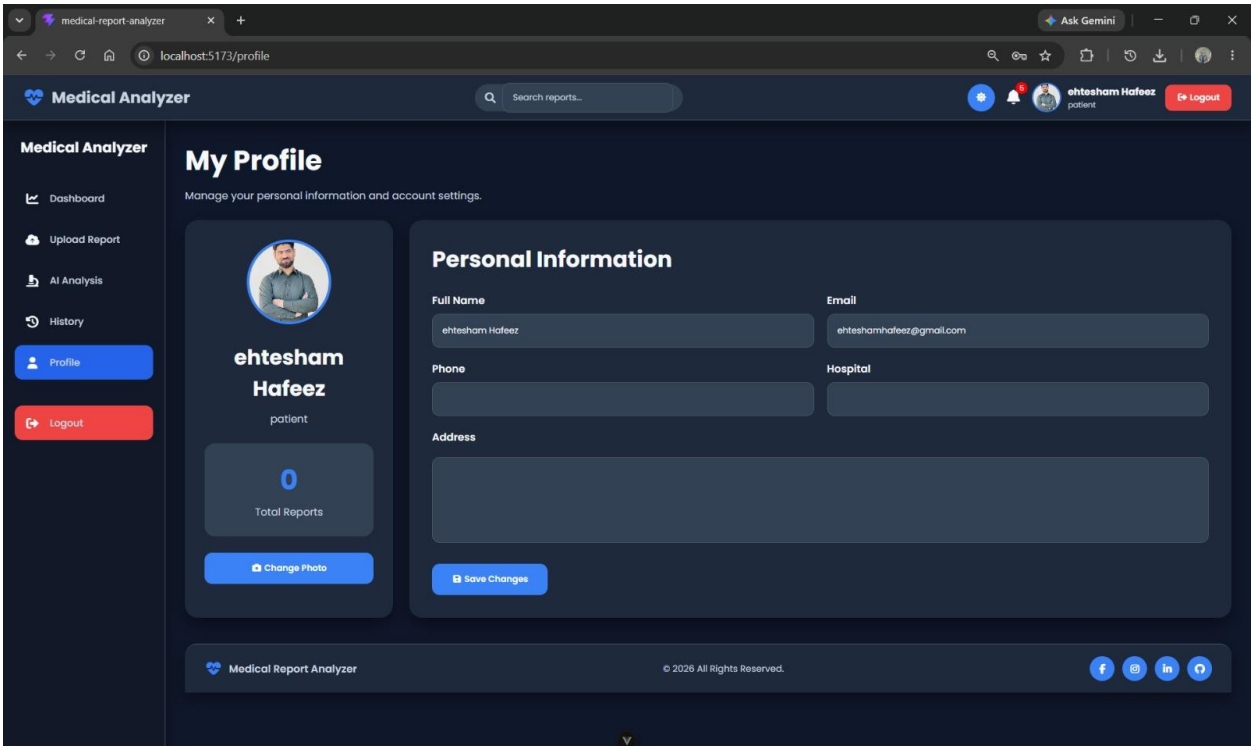
2.6 Analysis Page:



2.7 History Page:



2.8 Profile Page:



2. Backend (Built with Node.js + Express.js):

3.1 Backend Overview

The backend is developed using Node.js and Express.js. It provides REST APIs for user authentication, medical report management, AI analysis and database communication. The frontend communicates with these APIs using HTTP requests.

3.2 MVC Architecture

The backend follows the Model-View-Controller (MVC) architecture. Models define the database structure, Controllers contain business logic, Routes handle API endpoints, Middleware manages request validation and authentication, while configuration files manage database connectivity.

3.3 MongoDB and Mongoose

MongoDB is used as the database and Mongoose provides schema-based models including User.js and Report.js.

3.4 Authentication and Authorization

Authentication is implemented using JWT tokens. Passwords are securely hashed using bcrypt before storage. Protected routes use authMiddleware.js to verify user identity.

3.5 Controllers

authController.js manages login and registration while reportController.js handles report upload, retrieval and analysis.

3.6 Routes and APIs

Express routes expose RESTful APIs for registration, login, uploading reports, viewing history and fetching profile information.

3.7 File Uploads

The uploads folder stores uploaded medical reports before processing.

3.8 Gemini AI Integration

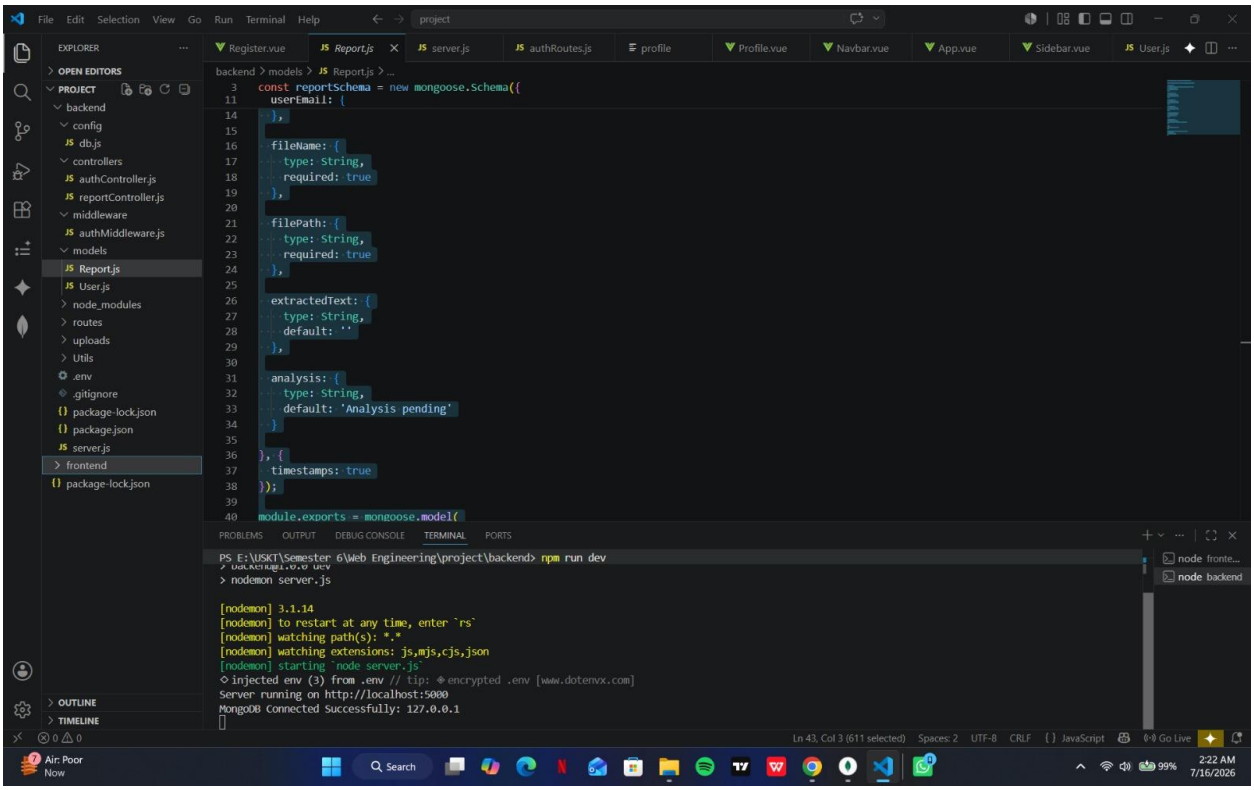
Utils/gemini.js integrates Google Gemini AI to generate report summaries and analysis.

3.9 Environment Variables

Sensitive configuration values such as database connection strings and JWT secrets are stored inside the .env file.

3.10 Backend Folder Structure

The project contains config, controllers, middleware, models, routes, uploads, Utils, package.json and .env folders/files to keep the backend modular and maintainable.



Conclusion

The backend complements the Vue frontend by providing secure authentication, database management, AI-powered report analysis and REST APIs using a scalable MVC architecture.